

```
import numpy as np
import pandas as pd
df= pd.read_csv(r"/content/TrainerRecommendation.csv")
(df.head())
```

	trainer_id	name	age	gender	height_cm	weight_kg	specialization	experience_years	location	rating	total_reviews
0	1	Fred Rodriguez	43	Female	181	90	Cardio	1	Chicago	1.3	1511
1	2	David Herring	51	Other	165	56	CrossFit	7	New York	1.6	160
2	3	Johnny Miles	43	Female	179	97	Weight Loss	25	Chicago	1.6	1743
3	4	Donna Weber	42	Female	175	113	CrossFit	24	Chicago	3.0	1448
4	5	Ronald Evans	49	Female	169	63	Muscle Gain	27	San Jose	1.8	1080

```
df.head(10)
```

	trainer_id	name	age	gender	height_cm	weight_kg	specialization	experience_years	location	rating	total_reviews
0	1	Fred Rodriguez	43	Female	181	90	Cardio	1	Chicago	1.3	1511
1	2	David Herring	51	Other	165	56	CrossFit	7	New York	1.6	160
2	3	Johnny Miles	43	Female	179	97	Weight Loss	25	Chicago	1.6	1743
3	4	Donna Weber	42	Female	175	113	CrossFit	24	Chicago	3.0	1448
4	5	Ronald Evans	49	Female	169	63	Muscle Gain	27	San Jose	1.8	1080
5	6	Heather Smith	44	Other	192	88	Muscle Gain	11	Phoenix	3.7	1128
6	7	Adam Guerra	55	Other	191	96	Muscle Gain	29	Seattle	4.9	78
7	8	Elizabeth Carter	28	Female	197	86	CrossFit	8	Dallas	2.8	1991
8	9	Catherine Castaneda	55	Female	185	116	Weight Loss	35	San Diego	1.5	1175

Next steps:

Generate code with df

View recommended plots

New interactive sheet

```
df['specialization'].value_counts()
```

	count
specialization	
Zumba	1975
Weight Loss	1900
Yoga	1898
Strength Training	1889
Cardio	1877
CrossFit	1823
Muscle Gain	1820
Pilates	1818

```
df.shape
```

```
(15000, 11)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   trainer_id          15000 non-null  int64
```

```
1  name          15000 non-null object
2  age           15000 non-null int64
3  gender        15000 non-null object
4  height_cm     15000 non-null int64
5  weight_kg     15000 non-null int64
6  specialization 15000 non-null object
7  experience_years 15000 non-null int64
8  location      15000 non-null object
9  rating        15000 non-null float64
10 total_reviews 15000 non-null int64
dtypes: float64(1), int64(6), object(4)
memory usage: 1.3+ MB
```

```
df.isnull().sum()
```



	0
trainer_id	0
name	0
age	0
gender	0
height_cm	0
weight_kg	0
specialization	0
experience_years	0
location	0
rating	0
total_reviews	0

dtypes: int64

```
df['rating'].value_counts()
```



	count
rating	
1.7	429
3.8	412
1.5	412
2.2	403
3.9	401
2.3	399
2.9	399
2.0	396
4.6	394
1.2	392
3.2	389
1.9	387
2.7	387
3.3	383
4.8	377
4.1	375
4.2	374
4.4	374
2.8	370
4.7	369
1.4	368
4.9	367
1.3	366
3.6	366
1.8	364
2.6	363
2.5	359
3.5	359
1.6	359
4.3	356
3.1	356
3.0	355
4.5	355
2.1	354
2.4	352
4.0	352
3.7	348
1.1	345
3.4	340
5.0	211
1.0	183

data: int64

```
# #this is trial version of the below code
# from sklearn.feature_extraction.text import TfidfVectorizer
```

```
# from sklearn.metrics.pairwise import cosine_similarity
# vectorizer= TfidfVectorizer()
# tfidf_matrix= vectorizer.fit_transform(df['combined_features'])
# cosine_mat= cosine_similarity(tfidf_matrix)
# cosine_mat.shape
# def recommend_trainers_by_input(location, specialization, min_rating=0.0, min_experience=0, top_n=5)
#     input_features = location.lower() + ' ' + specialization.lower()
#     input_vec = vectorizer.transform([input_features])
#     cosine_sim = cosine_similarity(input_vec, tfidf_matrix).flatten()
#     df['similarity'] = cosine_sim
#     filtered_df = df[(df['rating'] >= min_rating) & (df['experience_years'] >= min_experience)]
#     recommended = filtered_df.sort_values(by='similarity', ascending=False).head(top_n)
#     return recommended[['name', 'weight_kg', 'height_cm', 'age']]
```

Start coding or [generate](#) with AI.

#this is the original code .above is only a analysis of the data

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
class TrainerRecommender:
    def __init__(self, df):
        self.df = df.copy()
        self.df['combined_features'] = self.df['location'].str.lower() + ' ' + self.df['specialization'].str.lower()
        self.vectorizer = TfidfVectorizer()
        self.tfidf_matrix = self.vectorizer.fit_transform(self.df['combined_features'])

    def recommend(self, location, specialization, min_rating=0.0, min_experience=0, top_n=5):
        input_features = location.lower() + ' ' + specialization.lower()
        input_vec = self.vectorizer.transform([input_features])
        cosine_sim = cosine_similarity(input_vec, self.tfidf_matrix).flatten()

        self.df['similarity'] = cosine_sim
        filtered_df = self.df[(self.df['rating'] >= min_rating) & (self.df['experience'] >= min_experience)]
        recommended = filtered_df.sort_values(by='similarity', ascending=False).head(top_n)

        return recommended[['name', 'weight_kg', 'height_cm', 'age']]
```

```
import pickle
df = pd.read_csv(r"/content/TrainerRecommendation.csv")
df['specialization'] = df['specialization'].fillna('')
df['location'] = df['location'].fillna('')
df['rating'] = df['rating'].fillna(df['rating'].mean())
df['experience'] = df['experience_years'].fillna(0)
recommender = TrainerRecommender(df)
with open("trainer_recommender.pkl", "wb") as f:
    pickle.dump(recommender, f)
```

#this is test one.

```
recommender.recommend(location= "chicago",specialization="crossfit",min_rating= 4,min_experience=20)
```



	name	weight_kg	height_cm	age
14138	Robert Morgan	116	157	56
14405	Kayla Levy	113	181	52
648	Jesse Smith	63	184	38
5500	Melissa Roberts	104	194	52
2323	Renee Valentine	114	158	58




```
with open("trainer_recommender.pkl", "rb") as f:
    recommender = pickle.load(f)
result = recommender.recommend(
    location="chicago",
    specialization="crossfit",
    min_rating=4.0,
    min_experience=3,
    top_n=5
)
```